

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Grafos: problemas clásicos

Problema 1: Camino mínimo

En muchas aplicaciones buscamos la **ruta más eficiente** entre dos puntos teniendo en cuenta costos o tiempos.

Casos reales

- En logística: minimizar tiempo de entrega entre depósito y cliente.
- En navegación: encontrar la ruta con menor tiempo entre dos ciudades.
- En redes: enrutar paquetes minimizando latencia o costo.

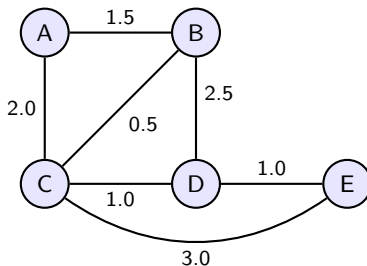
A continuación mostramos un ejemplo simple y cómo resolverlo con `networkx`.

Ejemplo: red de rutas entre ciudades

El archivo `rutas.csv` (sin encabezado) contiene aristas con peso (tiempo en horas):

```
A,B,1.5  
A,C,2.0  
B,C,0.5  
B,D,2.5  
C,D,1.0  
C,E,3.0  
D,E,1.0
```

Cada fila representa una ruta (origen,destino,tiempo). El grafo es no dirigido (las rutas permiten ida y vuelta).



Camino mínimo: concepto y algoritmos

Objetivo: dado un grafo con pesos y dos nodos origen y destino, encontrar el camino que minimiza la suma de pesos.

Algoritmos típicos

- **Dijkstra:** grafos con pesos no negativos.
- **Bellman-Ford:** permite pesos negativos (además detecta ciclos negativos).
- **BFS:** grafos no ponderados (o todos los pesos iguales).

En la práctica: para Dijkstra en `networkx` usamos `nx.shortest_path` y `nx.shortest_path_length`.

Camino mínimo: código

```
import networkx as nx
import matplotlib.pyplot as plt

def cargar_rutas(nombre_archivo):
    # Lee un edgelist ponderado (origen, destino, peso)
    G = nx.read_edgelist(
        nombre_archivo,
        delimiter=",",
        data=[("distancia", float)]
    )
    return G

G = cargar_rutas("rutas.csv")

# Camino mínimo desde A hasta E usando Dijkstra
camino = nx.shortest_path(G, source="A", target="E", weight="distancia")
dist = nx.shortest_path_length(G, source="A", target="E", weight="distancia")

print("Camino mínimo de A a E :", camino)
print("Distancia total (horas) :", dist)

# Dibujo (simple)
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_color="lightblue", node_size=1400)
labels = nx.get_edge_attributes(G, "distancia")
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
plt.show()
```

Camino mínimo con restricción de peso

En algunas aplicaciones operativas hay restricciones que impiden usar rutas demasiado largas o riesgosas. Por ejemplo, solo pueden utilizarse caminos cuya duración sea menor o igual que un umbral K .

Ejercicio 1. Se dispone de información sobre una red de rutas entre distintas localidades. Cada nodo representa una **localidad**, y una arista ponderada (u, v, w) indica que existe un camino entre u y v con un **tiempo de viaje** igual a w .

El archivo `rutas.csv` (sin encabezado) tiene el siguiente formato:

```
A,B,1.5
A,C,2.0
B,C,0.5
B,D,2.5
C,D,1.0
...
```

Se pide:

- Construir un grafo ponderado G a partir del archivo `rutas.csv`.
- Calcular el camino mínimo entre A y E .
- Implementar una función que, con $K = 1.5$, genere un subgrafo que incluya únicamente las aristas cuyo peso sea $\leq K$.
- Calcular el camino mínimo entre A y E utilizando únicamente ese subgrafo. ¿Coincide con el del grafo completo?

Problema 2: Assignment Problem

Motivación real

- Asignar trabajadores a tareas minimizando costos.
- Asignar máquinas a trabajos minimizando tiempos.
- Asignar repartidores a zonas minimizando distancia.
- Asignar profesores a cursos maximizando adecuación.

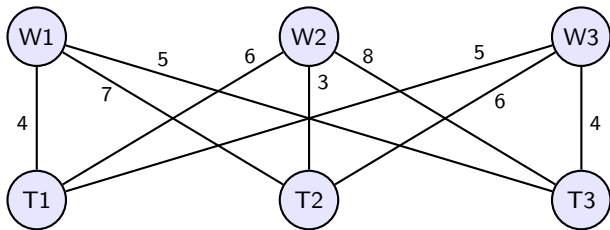
Estas aplicaciones llevan a buscar un **matching perfecto de costo mínimo** en un **grafo bipartito**: el **Assignment Problem**.

Ejemplo: trabajadores y tareas

El archivo `costos.csv` describe un grafo bipartito con costos de asignación:

```
W1,T1,4  
W1,T2,7  
W1,T3,5  
W2,T1,6  
W2,T2,3  
W2,T3,8  
W3,T1,5  
W3,T2,6  
W3,T3,4
```

Diagrama esquemático:



Assignment Problem

Objetivo: seleccionar asignaciones ($W \rightarrow T$) que:

- cubran todos los trabajadores y todas las tareas,
- sin que ningún trabajador o tarea se repita,
- minimizando la suma de costos.

Algoritmos clásicos

- **Algoritmo Húngaro:** método matricial.
- **Min-Cost Max-Flow:** modelar como red de flujo de costo mínimo.

En `networkx` usamos

```
nx.algorithms.bipartite.matching.minimum_weight_full_matching.
```

Assignment Problem: Código

```
import networkx as nx

def cargar_costos(nombre_archivo):
    G = nx.read_edgelist(
        nombre_archivo,
        delimiter=",",
        data=[("costo", float)]
    )
    return G

G = cargar_costos("costos.csv")

matching = nx.algorithms.bipartite.matching.minimum_weight_full_matching(G,
                                                                            weight="costo")

print("Asignaciones óptimas:")

costo_total = 0
trabajadores = ["W1", "W2", "W3"]
for u in trabajadores:
    v = matching[u]
    costo = G[u][v]["costo"]
    print(u, "<->", v, "costo:", costo)
    costo_total = costo_total + costo

print("Costo total mínimo:", costo_total)
```

`minimum_weight_full_matching` devuelve un diccionario `M` con los pares del matching perfecto de costo mínimo, duplicados `M[u]=v` y `M[v]=u`).

Ejercicio: asignación de consultores a proyectos

Ejercicio 2. Una empresa debe asignar **consultores** a **proyectos**. Cada arista (C_i, P_j) representa el **costo estimado** de asignar el consultor C_i al proyecto P_j .

El archivo `costos.csv` (sin encabezado) tiene el siguiente formato:

```
C1 , P1 , 4
C1 , P2 , 7
C1 , P3 , 5
C2 , P1 , 6
C2 , P2 , 3
...
```

Restricción especial:

- **C2 debe ser asignado a P3.**

Se pide:

- Cargar el archivo y construir un grafo bipartito ponderado G .
- Calcular el matching perfecto de costo mínimo **con** la restricción.
- Calcular el matching perfecto de costo mínimo **sin** la restricción.

¿El resultado de (a) y (b) fue el mismo?

Para resolver las dos versiones del problema (con y sin la restricción), puede resultar útil usar `G.copy()`. Este método crea una **copia independiente** del grafo original, permitiendo modificarla (eliminar aristas para imponer la restricción) sin alterar el grafo inicial.

Problema 3: Conectividad y puntos críticos

Motivación real

- Resiliencia de una red eléctrica: ¿qué subestaciones son críticas?
- Robustez de una red de comunicaciones frente a fallas de routers.
- En redes sociales: nodos cuya eliminación fragmenta la comunidad.

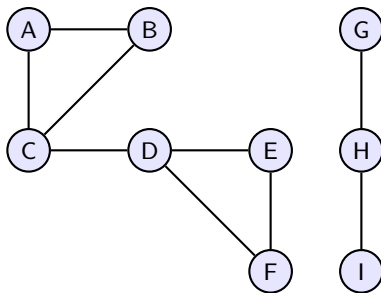
Identificar **componentes conexas**, **puentes** y **puntos de articulación** es clave para analizar vulnerabilidades.

Ejemplo: red de estaciones (CSV)

Archivo estaciones.csv (sin encabezado):

```
A,B  
A,C  
B,C  
C,D  
D,E  
E,F  
D,F  
G,H  
H,I
```

Aquí hay una componente grande (A..F) y una pequeña (G..I).



Conectividad: conceptos clave

- **Componentes conexas:** subconjuntos de nodos conectados por algún camino.
- **Puntos de articulación:** nodos cuya eliminación aumenta el número de componentes.
- **Puentes:** aristas cuya eliminación aumenta el número de componentes.

En `networkx`:

- `nx.connected_components(G)` para componentes.
- `nx.articulation_points(G)` para puntos de articulación.
- `nx.bridges(G)` para puentes.

Conectividad y fallas: código

```
import networkx as nx

def cargar_estaciones(nombre_archivo):
    G = nx.read_edgelist(nombre_archivo, delimiter=",")
    return G

G = cargar_estaciones("estaciones.csv")

# Componentes
componentes = list(nx.connected_components(G))
print("Componentes conexas :", componentes)

# Puntos de articulación
art_points = list(nx.articulation_points(G))
print("Puntos de articulación :", art_points)

# Puentes
puentes = list(nx.bridges(G))
print("Puentes :", puentes)

# Simular la falla de un nodo
def componentes_tras_falla(G, nodo):
    H = G.copy()
    if H.has_node(nodo):
        H.remove_node(nodo)
    return list(nx.connected_components(H))

print("Componentes tras falla de D :", componentes_tras_falla(G, "D"))
```

Ejercicio 3: Conectividad bajo fallas

Ejercicio 3. Una empresa logística opera una red de **centros de distribución** conectados entre sí por **rutas internas de transporte**. Cada nodo representa un centro, y cada arista indica que existe un corredor operativo entre dos centros.

La empresa quiere planificar mantenimientos y detectar vulnerabilidades que puedan provocar la **interrupción del flujo de mercadería** si falla un centro o un corredor clave.

Dado `estaciones.csv`:

- (a) Identificar si hay centros totalmente aislados
- (b) Determinar qué centros y/o rutas, de no estar funcionales, fragmentaría la red logística.
- (c) Para cada centro crítico, simular su falla y reportar cuántas subredes quedarían incomunicadas.

Resumen de la clase

Vimos tres problemas clásicos:

- Camino mínimo.
- MST.
- Conectividad.

Existen muchísimos otros problemas clásicos que pueden modelarse usando grafos, como el problema del viajante, el coloreo de mapas, la satisfacibilidad de fórmulas, entre muchos otros.

Con lo visto, pueden completar la sección 4 de la guía de Métodos Analíticos.